

Chapter 5 Array-Based Sequences

Low-Level Arrays

Prompt 1: What is computer memory made of?

✓ Answer:

Computer memory is made of **bits**, which are the smallest units of data (0 or 1).

Bits are grouped into larger units called **bytes**, where:

1 byte = 8 bits

Modern computers have billions of bytes of memory.

Prompt 2: What is a memory address? Why is it needed?

✓ Answer:

A **memory address** is a unique number assigned to each byte in memory.

It allows the computer to **locate and access data precisely**.

Without memory addresses, the computer would not know *where* data is stored.

🖼 Image: Memory with Addresses

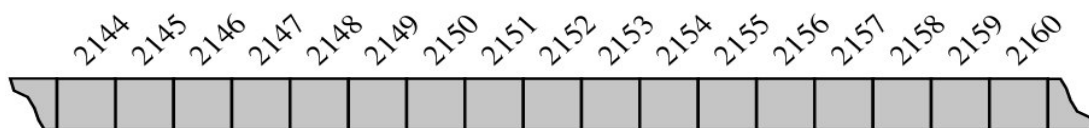


Figure 5.1: A representation of a portion of a computer's memory, with individual bytes labeled with consecutive memory addresses.

Each row represents **one byte** of memory.

Prompt 3: Why is main memory called Random Access Memory (RAM)?

✓ Answer:

Memory is called **RAM** because:

- Any memory address can be accessed **directly**

- Access time is the same no matter where the data is stored

✦ Retrieving byte #10 takes the same time as byte #8,675,309.

In algorithm analysis, this is written as:

O(1) time (constant time)

Prompt 4: How does a programming language use memory addresses?

✓ **Answer:**

A programming language keeps a **mapping** between:

- Variable names (identifiers)
- Memory addresses where their values are stored

Example:

```
x = 10
```

```
y = 20
```

Internally:

- $x \rightarrow$ some memory address
- $y \rightarrow$ another memory address

Prompt 5: Why do we need arrays instead of many variables?

✓ **Answer:**

Arrays allow us to store **multiple related values** under **one name**.

Example:

Instead of:

```
score1, score2, score3, ..., score10
```

We use:

```
scores[0], scores[1], scores[2], ..., scores[9]
```

This makes programs:

- Cleaner
- Easier to manage
- More scalable

Prompt 6: What is an array in low-level memory terms?

✓ Answer:

An **array** is a group of elements stored:

- In **contiguous (adjacent) memory locations**
- With **each element having the same size**

This layout enables fast access using index calculations.

🖼 Image: Array Stored in Memory

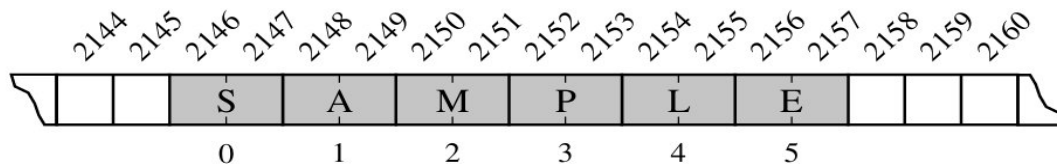


Figure 5.2: A Python string embedded as an array of characters in the computer's memory. We assume that each Unicode character of the string requires two bytes of memory. The numbers below the entries are indices into the string.

Each character occupies **equal space**.

Prompt 7: How are strings stored in Python at the memory level?

✓ Answer:

- Python strings are arrays of **Unicode characters**
- Each character typically uses **2 bytes (16 bits)**

Example string:

"SAMPLE"

- 6 characters
- 12 bytes total
- Stored contiguously in memory

Prompt 8: What is a cell in an array?

✓ Answer:

A **cell** is:

- One position in an array
- Holds exactly **one element**
- Identified by an **index**

Indices always start from **0**.

🖼 Image: Array Cells and Indices

Array →	S	A	M	P	L	E
Index →	0	1	2	3	4	5

Figure 5.3: A higher-level abstraction for the string portrayed in Figure 5.2.

Prompt 9: Why must all array elements use the same number of bytes?

✓ Answer:

This allows the computer to calculate the address of any element using a formula.

If sizes were different, the computer would have to scan the array one-by-one.

Uniform size $\Rightarrow$ **constant-time access**

Prompt 10: How does the computer compute the address of an array element?

✓ Answer:

The address is computed using:

$$address = start + (index \times cell\ size)$$

★ Example:

- Start address = **2146**
- Cell size = **2 bytes**
- Index = **4**

$$2146 + (4 \times 2) = 2146 + 8 = 2154$$

So element at index 4 starts at memory address **2154**.

Prompt 11: Why can array indexing be done in $O(1)$ time?

✓ Answer:

Because:

- The address is computed using arithmetic
- No looping or searching is needed

This makes arrays **extremely efficient** for accessing elements.

Prompt 12: What is the high-level view of arrays for programmers?

✓ Answer:

Programmers do **not** think about memory addresses.

Instead, they see arrays like this:

🖼 Image: High-Level Array View



Figure 5.3: A higher-level abstraction for the string portrayed in Figure 5.2.

The language runtime handles all memory calculations automatically.

Referential Arrays

Prompt 1: Why can't Python store strings directly in a low-level array?

✓ Answer:

Low-level arrays require that **every cell uses the same number of bytes**.

Strings:

- Have **different lengths**
- Require **different amounts of memory**

If Python tried to store strings directly:

- It would need to reserve space for the *largest possible string*
- This would waste huge amounts of memory

➡ **Conclusion:** Python cannot store variable-sized objects directly inside an array.

Prompt 2: How does Python solve the problem of storing variable-sized objects in arrays?

✓ Answer:

Python stores **references (memory addresses)** to objects, not the objects themselves.

- Each array cell stores a **fixed-size reference**
- The actual object is stored elsewhere in memory
- All references are the same size (e.g., 64 bits)

This design is called a **referential array**.

Prompt 3: What is a referential array?

✓ Answer:

A **referential array** is an array where:

- Each cell contains a **reference (pointer)** to an object
- The objects themselves live elsewhere in memory

Python lists and tuples are **referential arrays**.

Prompt 4: How does a list of patient names look in memory?

✓ Answer:

Each list cell stores a **reference** to a string object.

🖼 Image: Referential Array of Patient Names

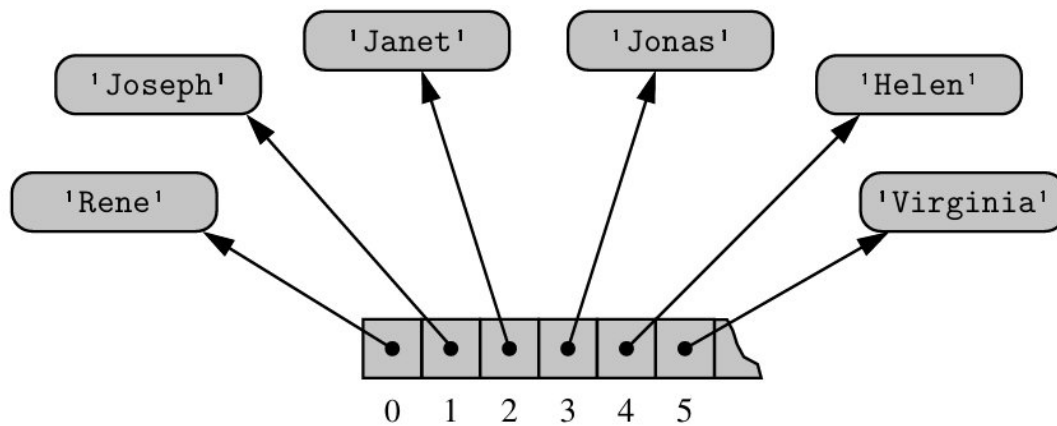


Figure 5.4: An array storing references to strings.

Prompt 5: Can list elements be objects other than strings?

✓ Answer:

Yes. A list can store references to:

- Strings
- Integers
- Custom objects (e.g., `Patient` instances)
- `None`

Example:

```
beds = [Patient(...), None, Patient(...)]
```

`None` can represent an **empty hospital bed**.

Prompt 6: What does it mean that lists are referential structures?

✓ Answer:

Lists store **references**, not actual objects.

This means:

- Multiple list entries can reference the **same object**
- Multiple lists can reference the **same object**

This behavior is called **aliasing**.

Prompt 7: What happens when a list is sliced?

✓ Answer:

Slicing creates:

- A **new list**
- But with **references to the same objects**

This is a **shallow copy**.

🖼 Image: List Slice Sharing References

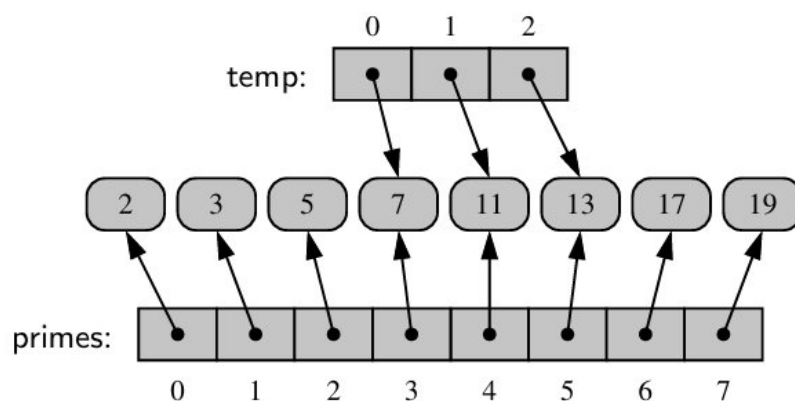


Figure 5.5: The result of the command `temp = primes[3:6]`.

Both lists point to the **same integer objects**.

Prompt 8: What happens when an element assignment is made in a sliced list?

✓ Answer:

The reference in the list is changed — **not the original object**.

🖼 Image: Assignment Changes Reference Only

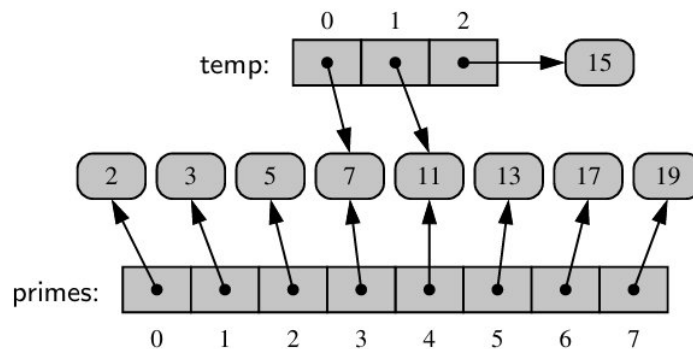


Figure 5.6: The result of the command `temp[2] = 15` upon the configuration portrayed in Figure 5.5.

Prompt 9: What is a shallow copy of a list?

✓ Answer:

A shallow copy of a list is a new list object whose elements are references to the same objects that appear in the original list.

This means:

- The list itself is new
- The elements inside are shared
- No new element objects are created

```
backup = list(primes)
```

What happens internally:

Python creates a new list

It copies references from primes

Both lists now point to the same element objects

Prompt 10: When is a deep copy needed?

✓ **Answer:**

A **deep copy** is needed when:

- List elements are **mutable**
- You want **completely independent objects**

Use:

```
from copy import deepcopy  
  
deep = deepcopy(original)
```

Prompt 11: What happens with list multiplication (`[0] * 8`)?

✓ **Answer:**

```
counters = [0] * 8
```

- Creates 8 references
- All reference the **same integer object**

🖼️ **Image: Aliasing with List Multiplication**

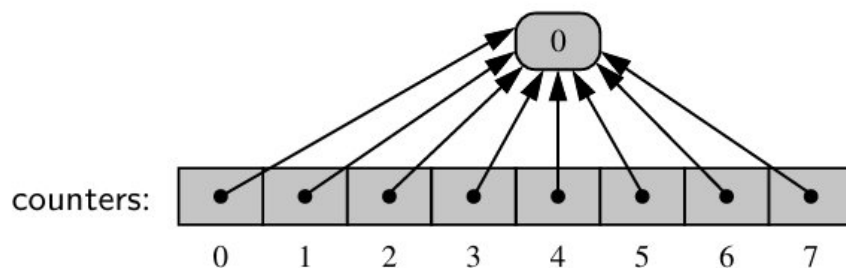


Figure 5.7: The result of the command `data = [0] * 8`.

All arrows point to the same 0.

Prompt 12: Why is this safe with integers?

✓ Answer:

Integers are **immutable**.

Operation:

```
counters[2] += 1
```

Actually does:

1. Create new integer 1
2. Update reference at index 2

🖼 Image: After counters[2] += 1

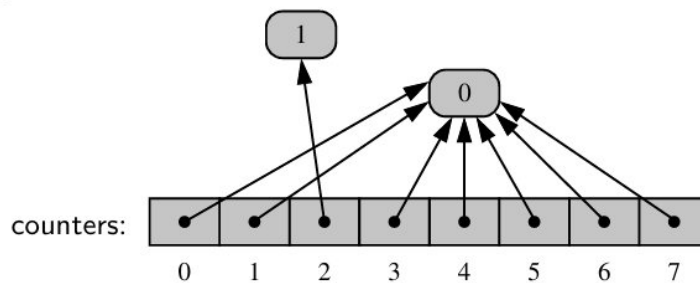


Figure 5.8: The result of command `data[2] += 1` upon the list from Figure 5.7.

Only index 2 changes.

Prompt 13: What does `list.extend()` do internally?

✓ Answer:

`extend()`:

- Appends **references** from another list
- Does **not copy objects**

Image: Effect of extend()

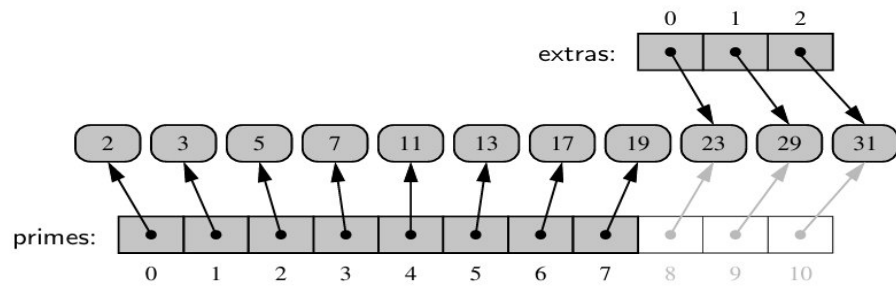


Figure 5.9: The effect of command `primes.extend(extras)`, shown in light gray.

Compact Arrays in Python

Prompt 1: What is a compact array in Python?

✓ Answer:

A **compact array** is an array that stores the **actual data values directly** in contiguous memory locations, rather than storing references to objects.

Examples:

- Python **strings**
- Arrays created using the **array module**

In a compact array:

- The bits representing the data are stored directly
- No extra memory is used for references

Prompt 2: How is a compact array different from a referential array?

✓ Answer:

Feature	Compact Array	Referential Array
Stores	Actual data bits	Memory addresses (references)
Element size	Fixed	Fixed (reference size)
Object storage	Inline	Stored elsewhere
Memory usage	Low	High
Example	str, array	list, tuple

Prompt 3: Why are Python strings considered compact arrays?

✓ Answer:

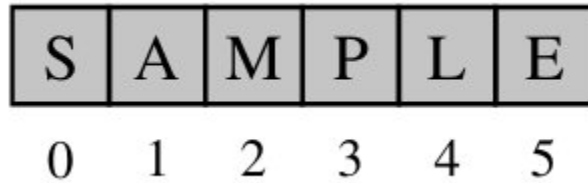
Python strings:

- Store characters **directly**
- Use an array of Unicode characters

- Each character typically uses **2 bytes**

There are **no references** to character objects.

Image: Compact Storage of a String



All characters are stored **consecutively** in memory.

Prompt 4: Why are compact arrays more memory-efficient?

Answer:

Compact arrays:

- Store **only the data**
- Do **not store references**
- Avoid per-object overhead

In contrast, referential arrays store:

- A reference (usually 64 bits)
- A separate object with its own overhead

Prompt 5: Why is storing characters as independent strings inefficient?

Answer:

If each character were stored as:

```
["A", "S", "M", "P", "L", "E"]
```

Then:

- Each character is a string object
- Each object has significant overhead
- Each list element stores a reference

This uses **many more bytes** than a compact string.

Prompt 6: Why do compact arrays improve computing performance?

✓ Answer:

Compact arrays store data:

- **Contiguously**
- In nearby memory locations

This improves:

- Cache locality
- CPU performance
- Sequential access speed

Prompt 7: Why are referential arrays worse for cache performance?

✓ Answer:

In referential arrays:

- References are contiguous
- Objects may be scattered across memory

This causes:

- Cache misses
- Slower access

Prompt 8: How inefficient is a Python list of integers?

✓ Answer:

For a list of **1 million integers**:

- Desired size:
 $64 \text{ bits} \times 1,000,000 = 8 \text{ MB}$
- Actual Python list usage:
 - ~8 bytes for reference
 - ~14 bytes for integer object

- ≈ 18 bytes per element

➡ 4–5× more memory than necessary

Prompt 9: What is `sys.getsizeof()` used for?

✓ Answer:

It reports the number of bytes used by an object's **primary storage**.

Example:

```
import sys

sys.getsizeof(10)
```

Typical result:

14 bytes

This shows the overhead of Python objects.

```
+-----+
| Reference Count      | ← 4 bytes
+-----+
| Type Pointer        | ← 4 bytes
+-----+
| Size / Sign         | ← 2 bytes
+-----+
| Value (10)          | ← 4 bytes
+-----+
```

Total ≈ 14 bytes

Prompt 10: What Python module supports compact arrays?

✓ Answer:

The **array module** provides compact storage for primitive data types.

```
from array import array
```

Prompt 11: What is the purpose of a type code in the array module?

✓ Answer:

A type code:

- Specifies the data type
- Determines how many bytes each element uses
- Allows compact storage

Example:

```
array('i', [2, 3, 5])
```

Here 'i' means **signed integer**.

Prompt 12: How does an array of integers look in memory?

🖼 Image: Compact Integer Array (Figure 5.10)

2	3	5	7	11	13	17	19
0	1	2	3	4	5	6	7

Figure 5.10: Integers stored compactly as elements of a Python array.

- No references
- All integers stored inline
- Fixed-size elements

Prompt 13: How do array objects differ from lists in usage?

✓ Answer:

Similarities:

- Indexing
- Iteration
- Length

Differences:

- Array requires **type code**
- Only stores **primitive types**
- More memory-efficient
- Less flexible

Prompt 14: What are the common type codes supported by the array module?

✓ **Answer (Simplified):**

Code	Meaning	Bytes (Typical)
b	signed char	1
B	unsigned char	1
u	Unicode char	2–4
h	short int	2
i	int	2–4
l	long int	4
f	float	4
d	double	8

Dynamic Arrays and Amortization

Prompt 1: Why must the size of a low-level array be fixed at creation time?

✓ **Answer:**

Low-level arrays must be created with a **fixed size** because:

- Memory must be allocated as a **contiguous block**
- Neighboring memory locations may already be in use
- The system cannot safely “grow” an array in place

Once allocated, an array **cannot expand** without moving.

🖼 **Image: Fixed-Size Array in Memory (Figure 5.11)**

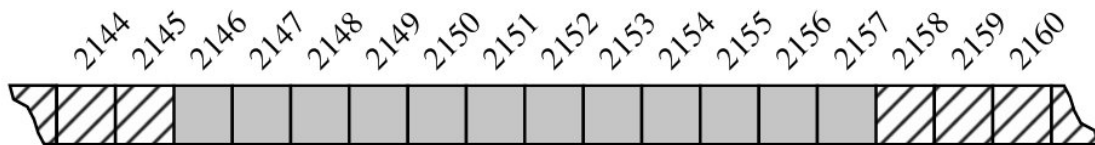


Figure 5.11: An array of 12 bytes allocated in memory locations 2146 through 2157.

This block is reserved entirely for the array.

Prompt 2: Why is fixed-size memory not a problem for strings and tuples?

✓ **Answer:**

Because **strings and tuples are immutable**.

- Their size never changes after creation
- Python knows the exact size in advance
- The underlying array can be allocated once and forever

Prompt 3: Why is fixed-size memory a problem for Python lists?

✓ Answer:

Python lists are **mutable**:

- Elements can be added or removed
- The final size is not known in advance
- Users expect `append()` to work indefinitely

Yet low-level arrays cannot grow naturally.

Prompt 4: How does Python solve this problem?

✓ Answer:

Python uses a technique called a **dynamic array**.

A dynamic array:

- Uses a low-level array internally
- Allocates **extra unused capacity**
- Grows by allocating a new, larger array when needed

Prompt 5: What is the key idea behind a dynamic array?

✓ Answer:

A list maintains:

- A **length** (number of actual elements)
- A **capacity** (size of the underlying array)

The capacity is usually **greater than the length**.

Prompt 6: What happens when there is still extra capacity?

✓ Answer:

When capacity is available:

- `append()` places the new element in the next free cell
- No resizing is needed
- Operation runs in **O(1)** time

Prompt 7: What happens when capacity is exhausted?

✓ Answer:

When the array is full:

1. Python allocates a **larger array**
2. Copies all existing references into it
3. Reclaims the old array
4. Appends the new element

This operation takes $O(n)$ time.

Prompt 8: Why doesn't Python resize the array by just one cell?

✓ Answer:

Resizing by one each time would make every append expensive.

Instead, Python:

- Grows the array by a **larger factor**
- Spreads out the cost of resizing

This leads to **amortized constant time**.

Prompt 9: What does `sys.getsizeof()` measure for a list?

✓ Answer:

`sys.getsizeof(list)` reports:

- Memory used by the list's **internal structure**
- The underlying array
- Instance variables

It does **not include** memory used by elements stored in the list.

Prompt 10: What does Code Fragment 5.1 test?

```
1 import sys                                # provides getsizeof function
2 data = []
3 for k in range(n):                        # NOTE: must fix choice of n
4     a = len(data)                          # number of elements
5     b = sys.getsizeof(data)               # actual size in bytes
6     print('Length: {0:3d}; Size in bytes: {1:4d}'.format(a, b))
7     data.append(None)                     # increase length by one
```

Code Fragment 5.1: An experiment to explore the relationship between a list's length and its underlying size in Python.

✓ Answer:

The experiment:

- Appends elements one by one to a list
- Prints:
 - Logical length
 - Actual memory size

This reveals how capacity changes internally.

Length:	0;	Size in bytes:	72
Length:	1;	Size in bytes:	104
Length:	2;	Size in bytes:	104
Length:	3;	Size in bytes:	104
Length:	4;	Size in bytes:	104
Length:	5;	Size in bytes:	136
Length:	6;	Size in bytes:	136
Length:	7;	Size in bytes:	136
Length:	8;	Size in bytes:	136
Length:	9;	Size in bytes:	200
Length:	10;	Size in bytes:	200
Length:	11;	Size in bytes:	200
Length:	12;	Size in bytes:	200
Length:	13;	Size in bytes:	200
Length:	14;	Size in bytes:	200
Length:	15;	Size in bytes:	200
Length:	16;	Size in bytes:	200
Length:	17;	Size in bytes:	272
Length:	18;	Size in bytes:	272
Length:	19;	Size in bytes:	272
Length:	20;	Size in bytes:	272
Length:	21;	Size in bytes:	272
Length:	22;	Size in bytes:	272
Length:	23;	Size in bytes:	272
Length:	24;	Size in bytes:	272
Length:	25;	Size in bytes:	272
Length:	26;	Size in bytes:	352

Code Fragment 5.2: Sample output from the experiment of Code Fragment 5.1.

Prompt 11: Why does an empty list already use memory?

✓ **Answer:**

Even an empty list is an object and stores:

- Type information
- Reference count
- Length
- Capacity
- Pointer to underlying array

On the sample system:

Empty list → 72 bytes

Prompt 12: What internal state does a list maintain?

✓ Answer:

Although Python hides implementation details, a list conceptually maintains three key pieces of state:

`n` → number of actual elements in the list
`capacity` → maximum elements the underlying array can hold
`A` → reference to the underlying array

Why each is needed:

- **n (length)**
Allows Python to know how many valid elements exist
- **capacity**
Determines whether `append()` can occur without resizing
- **A (array reference)**
Points to the block of memory storing element references

This internal structure enables Python lists to:

- Grow dynamically
- Support fast indexing
- Achieve amortized $O(1)$ append performance

Prompt 13: Why does memory jump from 72 to 104 bytes after first append?

✓ Answer:

The increase:

$104 - 72 = 32$ bytes

On a 64-bit machine:

- Each reference = 8 bytes
- 32 bytes = space for 4 references

So Python allocates an array of capacity 4.

Before append:

[List object only]

72 bytes

After append:

[List object] + [Array of 4 references]

72 bytes + 32 bytes = 104

Prompt 14: Why doesn't memory increase for the 2nd, 3rd, and 4th append?

✓ **Answer:**

Because:

- Capacity = 4
- Appending within capacity needs no resizing

Memory stays constant.

Prompt 15: Why does memory jump at length = 5?

✓ **Answer:**

The array is full.

Python:

- Allocates a new array with capacity 8
- Uses 64 bytes for references (8×8)
- Total size becomes:

$72 + 64 = 136$ bytes

Prompt 16: How do later capacity increases behave?

✓ **Answer:**

Observed pattern:

Length	Capacity
1-4	4
5-8	8

9–16	16
17–25	25

Capacity grows **nonlinearly**, not by +1.

Prompt 17: Why does Python grow capacity more than doubling sometimes?

✓ Answer:

Python uses a tuned growth strategy to:

- Reduce reallocations
- Balance memory usage
- Improve real-world performance

Exact growth rules are implementation-specific.

Prompt 18: Why does appending `None` not affect memory size?

✓ Answer:

Because lists store **references**.

- Every element is just a reference
- Type of object doesn't matter
- `None`, `int`, or `string` → same reference size

Implementing a Dynamic Array Although the Python list class provides a highly

Prompt 1: Why can't an array grow in place?

✓ **Answer:**

A low-level array:

- Is allocated in **consecutive memory locations**
- Has a **fixed capacity**
- Cannot expand because adjacent memory may already be in use

Therefore, to “grow” an array, we must:

- Allocate a **new, larger array**
- Copy elements
- Discard the old array

Prompt 2: What happens when a dynamic array becomes full?

✓ **Answer:**

When an element is appended and the array is full, Python performs these steps:

1. Allocate a **new array B** with larger capacity
2. Copy all elements from old array **A** $\rightarrow$ **B**
3. Reassign reference **A** = **B**
4. Insert the new element
5. Old array becomes garbage-collected

Diagram (Figure 5.12 idea):

An illustration of this process is shown in Figure 5.12.

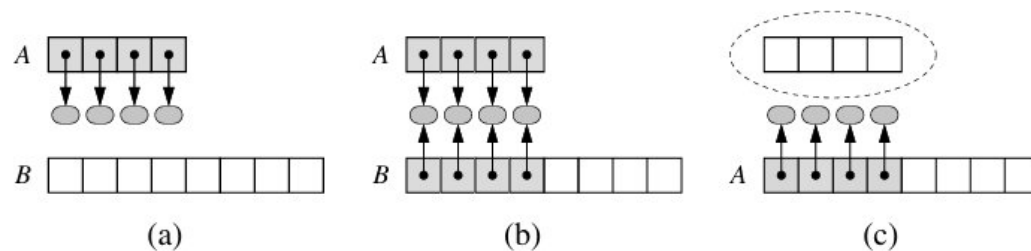


Figure 5.12: An illustration of the three steps for “growing” a dynamic array: (a) create new array *B*; (b) store elements of *A* in *B*; (c) reassign reference *A* to the new array. Not shown is the future garbage collection of the old array, or the insertion of the new element.

Prompt 3: Why does Python usually double the capacity?

✓ Answer:

Doubling the capacity:

- Reduces frequency of resizing
- Keeps `append()` fast on average
- Leads to **amortized $O(1)$** time complexity

If capacity increased by only 1 each time:

- Every `append` would require copying
- `Append` would be **$O(n)$** every time (very slow)

Prompt 4: What is the purpose of the `DynamicArray` class?

✓ Answer:

The `DynamicArray` class:

- Demonstrates how Python lists work internally
- Uses a **low-level array** for storage
- Manages:
 - `Size (n)`

- Capacity
- Resizing logic

It is **educational**, not meant to replace Python lists.

Prompt 5: What does `__init__` do in `DynamicArray`?

✓ **Answer:**

```
def __init__(self):
    self.n = 0

    self.capacity = 1

    self.A = self.make_array(self.capacity)
```

- Initializes an empty array
- Starts with **capacity = 1**
- Allocates low-level storage

Prompt 6: How does `append()` work?

✓ **Answer:**

```
def append(self, obj):
    if self.n == self.capacity:
        self._resize(2 * self.capacity)

    self.A[self.n] = obj

    self.n += 1
```

Steps:

1. Check if array is full
2. Resize if necessary
3. Insert new element
4. Increase size counter

Prompt 7: What does `_resize()` do?

✓ Answer:

```
def _resize(self, c):  
    B = self.make_array(c)  
    for k in range(self.n):  
        B[k] = self.A[k]  
    self.A = B  
    self.capacity = c
```

It:

- Creates a new larger array
- Copies elements
- Updates reference and capacity

Prompt 9: Why is copying necessary during resize?

✓ Answer:

Because:

- Old array memory cannot be expanded
- Data must be preserved
- New array starts empty

Copying ensures **data integrity**.

Prompt 9: What is `make_array()` and why is `ctypes` used?

✓ Answer:

```
return (c * ctypes.py_object)()
```

- `ctypes` allows creation of **raw low-level arrays**
- Stores references like Python lists do
- Mimics real internal list behavior

Python lists themselves are implemented in **C**, not Python.

Prompt 10: Why is the old array eventually removed?

✓ **Answer:**

Once `A` is reassigned:

- No references remain to old array
- Python's garbage collector reclaims the memory